

The Python **time** Module

A practical, interview-ready reference covering clocks, timestamps, sleeping, formatting, and `struct_time` — with fully commented code examples.

The **time** module is one of Python's core standard-library modules for working with time. It gives you access to the system clock, lets you pause execution, and lets you convert between raw timestamps and human-readable date/time strings. It is lower-level than the **datetime** module: it mostly deals with plain numbers (seconds since the epoch) rather than rich date objects.

1. The Epoch and `time.time()`

Most functions in this module measure time as the number of seconds that have passed since the **Unix epoch**: January 1, 1970, 00:00:00 UTC. This single floating-point number is called a **timestamp**.

```
import time # Import the standard library time module

# time.time() returns the current timestamp:
# a float representing seconds since the Unix epoch (Jan 1, 1970, UTC)
current_timestamp = time.time()

print(current_timestamp)
# Example output: 1795000123.456789

# Timestamps are great for measuring durations because they are
# just plain numbers you can subtract from each other.
start = time.time()      # Record the start time
# ... some work happens here ...
end = time.time()        # Record the end time
elapsed_seconds = end - start # Subtract to get the duration
print(f"Elapsed: {elapsed_seconds:.4f} seconds")
```

2. Pausing Execution: `time.sleep()`

`time.sleep(seconds)` suspends (pauses) the calling thread for the given number of seconds. It accepts a float, so sub-second delays like 0.5 are valid.

```

import time # Needed for sleep()

print("Starting task...") # Message before the pause

time.sleep(2) # Pause execution for 2 seconds
# Nothing else runs on this thread while sleep() is active

print("2 seconds have passed, task resumes.") # Message after the pause

# Sub-second sleeps are also valid:
time.sleep(0.25) # Pause for a quarter of a second

```

Tip: `time.sleep()` blocks the entire thread. In async code, use `asyncio.sleep()` instead so other tasks can keep running.

3. Measuring Durations Accurately

`time.time()` reflects the system's wall clock, which can jump backward or forward if the OS clock is adjusted (e.g., via NTP sync). For measuring elapsed time reliably, prefer `time.perf_counter()` or `time.monotonic()`.

```

import time # Contains perf_counter() and monotonic()

# perf_counter() uses the highest-resolution clock available
# and is ideal for benchmarking short code blocks.
start = time.perf_counter() # High-resolution start marker

total = sum(range(1_000_000)) # Some work to time

end = time.perf_counter() # High-resolution end marker
print(f"Sum took {end - start:.6f} seconds")

# monotonic() is guaranteed to never go backwards,
# making it safer than time.time() for measuring elapsed durations
# in long-running programs (e.g., timeouts, retries).
m_start = time.monotonic() # Monotonic start marker
time.sleep(1) # Simulate work
m_end = time.monotonic() # Monotonic end marker
print(f"Monotonic elapsed: {m_end - m_start:.4f} seconds")

```

Clock function comparison

Function	Use case	Can go backward?
<code>time.time()</code>	Wall-clock timestamp (dates, logging)	Yes (NTP sync)
<code>time.perf_counter()</code>	Precise benchmarking of code	No
<code>time.monotonic()</code>	Measuring elapsed time safely	No
<code>time.process_time()</code>	CPU time used by the process	No

4. struct_time: Breaking a Timestamp Into Fields

A raw timestamp is just a number. To read year, month, day, etc. you convert it into a **struct_time** object using `time.localtime()` (local timezone) or `time.gmtime()` (UTC).

```
import time # Provides localtime(), gmtime(), and struct_time

now_timestamp = time.time() # Current timestamp as a float

# Convert to a struct_time in the LOCAL timezone
local_struct = time.localtime(now_timestamp)
print(local_struct)
# time.struct_time(tm_year=2026, tm_mon=9, tm_mday=7, tm_hour=14,
#                  tm_min=30, tm_sec=5, tm_wday=0, tm_yday=250, tm_isdst=0)

# Convert to a struct_time in UTC
utc_struct = time.gmtime(now_timestamp)

# Individual fields can be accessed by name or by index
print(local_struct.tm_year) # e.g. 2026 (named access)
print(local_struct[0])     # e.g. 2026 (index access, same value)
print(local_struct.tm_hour) # Hour of the day, 0-23
print(local_struct.tm_wday) # Day of week, Monday = 0 ... Sunday = 6
```

5. Formatting: strftime() and strptime()

`time.strftime()` turns a `struct_time` into a formatted string. `time.strptime()` does the reverse: it parses a string into a `struct_time`. Both use the same set of format codes (directives).

```
import time # Provides strftime() and strptime()

struct_now = time.localtime() # Current time as a struct_time

# strftime(format, struct_time) -> formatted string
formatted = time.strftime("%Y-%m-%d %H:%M:%S", struct_now)
print(formatted) # e.g. "2026-09-07 14:30:05"

readable = time.strftime("%A, %B %d, %Y", struct_now)
print(readable) # e.g. "Monday, September 07, 2026"

# strptime(string, format) -> struct_time
# The string and format MUST match exactly, or a ValueError is raised
parsed = time.strptime("2026-09-07 14:30:05", "%Y-%m-%d %H:%M:%S")
print(parsed.tm_year, parsed.tm_mon, parsed.tm_mday) # 2026 9 7
```

Common format directives

Code	Meaning	Example
%Y	4-digit year	2026
%m	2-digit month	09

%d	2-digit day of month	07
%H	Hour, 24-hour clock	14
%M	Minute	30
%S	Second	05
%A	Full weekday name	Monday
%B	Full month name	September
%j	Day of the year (001-366)	250

6. Quick Human-Readable Strings

For a fast, no-formatting-required readable string, use `time.ctime()` (takes a timestamp) or `time.asctime()` (takes a `struct_time`).

```
import time # Provides ctime() and asctime()

# ctime() converts a timestamp directly into a readable string
print(time.ctime(time.time()))
# Example: "Mon Sep  7 14:30:05 2026"

# ctime() with no argument uses the current time automatically
print(time.ctime())

# asctime() does the same thing but takes a struct_time instead
print(time.asctime(time.localtime()))
# Example: "Mon Sep  7 14:30:05 2026"
```

7. Converting struct_time Back to a Timestamp

`time.mktime()` takes a `struct_time` in LOCAL time and converts it back into a timestamp (float). It is effectively the inverse of `time.localtime()`.

```
import time # Provides mktime()

# Build a struct_time for a specific date/time using a tuple:
# (year, month, day, hour, minute, second, weekday, yearday, isdst)
# weekday, yearday can be set to 0; isdst=-1 means "let Python figure it out"
custom_struct = time.struct_time((2026, 9, 7, 14, 30, 0, 0, 0, -1))

# mktime() converts that struct_time back into a Unix timestamp
timestamp = time.mktime(custom_struct)
print(timestamp) # e.g. 1794929400.0

# Round-trip check: localtime(mktime(x)) should reproduce x
round_trip = time.localtime(timestamp)
print(round_trip.tm_year, round_trip.tm_mon, round_trip.tm_mday) # 2026 9 7
```

8. Interview-Ready Summary

Key points that commonly come up when discussing the time module:

What is the epoch?

Jan 1, 1970, 00:00:00 UTC — the reference point all timestamps count from.

`time.time()` vs `time.perf_counter()`

`time.time()` is the wall clock (can jump); `perf_counter()` is monotonic and high-resolution, ideal for benchmarking.

`struct_time`

A named tuple-like object holding year, month, day, hour, minute, second, weekday, yearday, and isdst.

`localtime()` vs `gmtime()`

`localtime()` applies the system's local timezone offset; `gmtime()` always returns UTC.

`strftime()` vs `strptime()`

`strftime` formats a `struct_time` INTO a string; `strptime` parses a string INTO a `struct_time`.

`time` module vs `datetime` module

`time` deals mostly in raw floats/`struct_time` and is closer to the OS clock; `datetime` provides richer objects (timedelta arithmetic, timezone-aware objects) built for date logic.

End of guide — every code sample above is fully runnable and commented line-by-line for quick review.